CSD624

MASTER OF SCIENCE IN COMPUTER SCIENCE

Advanced Database Management in MS SQL

Dr A.J. Fofanah

Course Instructor | Advanced Database Management

| SQL Code | Advanced Techniques | Practical Examples

Advanced Master-Level Content

01 **Query Optimization & Execution Plans**

Execution plans · Statistics · SARGability · Hints · Parameter sniffing · AQP

02 **Advanced Indexing Strategies**

B-Tree internals · Covering · Columnstore · Filtered · Fragmentation · DMVs

03 **Security, Encryption & Compliance**

TDE · Always Encrypted · RLS · DDM · Audit · CDC · Backup encryption

04 **High Availability & Cloud Integration**

Always On AG · Log Shipping · Azure SQL MI · Query Store · In-Memory OLTP

05 **Advanced T-SQL Patterns**

Window functions · Recursive CTEs · Dynamic SQL · Error handling

SECTION 01

Query Optimization & Execution Plans

01

CSD624 · Advanced Database Management in MS SQL

Dr A.J. Fofanah

How SQL Server Processes a Query

Parsing

Syntax check → parse tree generation. T-SQL is validated against grammar rules.

Algebrization

Resolves object names, data types, and produces a logical operator tree (algebraized tree).

Optimization

The Query Optimizer searches for the lowest-cost execution plan using statistics and heuristics.

Execution

The Storage Engine executes the chosen plan, accessing pages via buffer pool or disk I/O.

Viewing Execution Plans

Tips

Use SET SHOWPLAN_XML to capture the execution plan as XML before running the query. The graphical plan in SSMS reveals estimated rows, cost percentages, and operator types. Always compare estimated vs actual row counts, large discrepancies indicate stale statistics.

```
-- Enable XML execution plan (no actual execution)
SET SHOWPLAN_XML ON;
GO

SELECT o.OrderID, c.CustomerName, SUM(od.Quantity * od.UnitPrice) AS Total
FROM Orders o
JOIN Customers c ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON od.OrderID = o.OrderID
WHERE o.OrderDate >= '2023-01-01'
GROUP BY o.OrderID, c.CustomerName;
GO

SET SHOWPLAN_XML OFF;
GO

-- Actual execution plan with runtime stats
SELECT *
FROM Orders o
JOIN Customers c ON c.CustomerID = o.CustomerID
OPTION (QUERYTRACEON 9481);  -- Force legacy CE
```

Reading Operator Costs

Tips

Each operator in the plan shows estimated CPU and I/O cost. The Thick arrow = more rows flowing. A Table Scan is almost always a red flag on large tables. Hash Match joins are used when no supporting index exists. Nested Loop joins are efficient for small outer inputs with indexed inner tables.

```sql
-- Force a specific join type to compare plans
SELECT c.CustomerName, COUNT(o.OrderID) AS OrderCount
FROM Customers c
LEFT JOIN Orders o ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerName
OPTION (LOOP JOIN);    -- Force nested loop

-- Then compare with:
OPTION (HASH JOIN);    -- Force hash join
OPTION (MERGE JOIN);   -- Force merge join (requires sorted inputs)

-- Identify expensive queries from the plan cache
SELECT TOP 10
    qs.total_elapsed_time / qs.execution_count AS avg_elapsed_us,
    qs.total_logical_reads / qs.execution_count AS avg_reads,
    SUBSTRING(qt.text, (qs.statement_start_offset/2)+1, 200) AS query_text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
ORDER BY avg_elapsed_us DESC;
```

Statistics & Cardinality Estimation

Tips

The Query Optimizer relies on column statistics (histograms) to estimate row counts. Outdated statistics cause poor plan choices — wrong join types, memory under/over-grants. UPDATE STATISTICS with FULLSCAN rebuilds histograms using all rows instead of the default 20-30% sample.

```
-- View statistics histogram for a column
DBCC SHOW_STATISTICS ('Sales.Orders', 'IX_Orders_CustomerID');

-- Update statistics with full scan (most accurate)
UPDATE STATISTICS Sales.Orders WITH FULLSCAN;

-- Update all statistics in the database
EXEC sp_updatestats;

-- Auto-update statistics threshold check
SELECT name, auto_update_statistics, auto_update_statistics_async
FROM sys.databases WHERE name = DB_NAME();

-- Find tables with stale statistics
SELECT
    OBJECT_NAME(s.object_id) AS table_name,
    s.name AS stat_name,
    sp.last_updated,
    sp.rows,
    sp.rows_sampled,
    CAST(100.0 * sp.rows_sampled / NULLIF(sp.rows,0) AS DECIMAL(5,2)) AS pct_sampled
FROM sys.stats s
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
WHERE OBJECT_SCHEMA_NAME(s.object_id) = 'Sales'
ORDER BY sp.last_updated ASC;
```

SARGable Predicates

Tips

A SARGable (Search ARGument able) predicate allows SQL Server to use an index seek rather than a full scan. Functions applied to indexed columns break SARGability. Implicit conversions (comparing INT to VARCHAR) also prevent seeks. Always design WHERE clauses to operate on raw column values.

```sql
-- ✗ NON-SARGable — forces Index Scan (function on column)
SELECT * FROM Orders WHERE YEAR(OrderDate) = 2023;
SELECT * FROM Orders WHERE LEFT(CustomerCode, 3) = 'ABC';
SELECT * FROM Orders WHERE CONVERT(VARCHAR, OrderID) = '1001';

-- ✅ SARGable equivalents — enables Index Seek
SELECT * FROM Orders
WHERE OrderDate >= '2023-01-01' AND OrderDate < '2024-01-01';

SELECT * FROM Orders WHERE CustomerCode LIKE 'ABC%';

SELECT * FROM Orders WHERE OrderID = 1001;  -- keep types consistent

-- ✗ Implicit conversion (CustomerID is INT, passing VARCHAR)
SELECT * FROM Orders WHERE CustomerID = '42';

-- ✅ Correct — explicit matching types
SELECT * FROM Orders WHERE CustomerID = 42;

-- Diagnose implicit conversions
SELECT * FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
WHERE CAST(qp.query_plan AS NVARCHAR(MAX)) LIKE '%CONVERT_IMPLICIT%';
```

Query Hints & Plan Guides

Tips

Query hints override the optimizer's decisions. Use sparingly, they can prevent the optimizer from adapting to data changes. OPTION(RECOMPILE) forces a new plan each execution (good for parameter sniffing issues). Plan guides let you attach hints to queries you cannot modify in application code.

```
-- RECOMPILE: generate fresh plan each time (fixes parameter sniffing)
SELECT * FROM Orders WHERE CustomerID = @CustID
OPTION (RECOMPILE);

-- OPTIMIZE FOR: pin plan to a specific value
SELECT * FROM Orders WHERE CustomerID = @CustID
OPTION (OPTIMIZE FOR (@CustID = 100));

-- OPTIMIZE FOR UNKNOWN: ignore supplied value, use average stats
SELECT * FROM Orders WHERE CustomerID = @CustID
OPTION (OPTIMIZE FOR (@CustID UNKNOWN));

-- MAXDOP hint: limit parallelism for this query
SELECT COUNT(*) FROM BigFactTable
OPTION (MAXDOP 1);

-- Create a plan guide for a query you cannot alter
EXEC sp_create_plan_guide
    @name = N'PG_Orders_CustID',
    @stmt = N'SELECT * FROM Orders WHERE CustomerID = @CustID',
    @type = N'SQL',
    @module_or_batch = NULL,
    @params = N'@CustID INT',
    @hints = N'OPTION (OPTIMIZE FOR (@CustID UNKNOWN))';
```

Parameter Sniffing — Diagnosis & Fix

Tips

Parameter sniffing occurs when SQL Server compiles a plan optimized for the first parameter value, then reuses it for all subsequent calls with different values. A plan good for 1 row can be catastrophic for 1 million rows. The key DMVs are sys.dm_exec_cached_plans and sys.dm_exec_query_stats.

```
-- Identify parameter sniffing victims in plan cache
SELECT
    qs.execution_count,
    qs.total_logical_reads,
    qs.min_logical_reads,
    qs.max_logical_reads,  -- huge gap = sniffing suspect
    SUBSTRING(qt.text, 1, 300) AS query_text,
    qp.query_plan
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
WHERE qs.max_logical_reads > qs.min_logical_reads * 100
ORDER BY qs.max_logical_reads DESC;

-- Solution: local variable technique (breaks sniffing)
CREATE PROCEDURE GetOrdersByCustomer @CustID INT AS
BEGIN
    DECLARE @LocalCustID INT = @CustID;  -- copy to local var
    SELECT * FROM Orders WHERE CustomerID = @LocalCustID;
END;

-- Or use RECOMPILE at procedure level
CREATE PROCEDURE GetOrdersByCustomer2 @CustID INT
WITH RECOMPILE
AS
    SELECT * FROM Orders WHERE CustomerID = @CustID;
```

Adaptive Query Processing (SQL 2019+)

Tips

Adaptive Query Processing (AQP) and Intelligent Query Processing (IQP) allow the optimizer to adjust plans at runtime. Batch Mode Adaptive Joins switch between Hash Join and Nested Loop after measuring actual rows. Memory Grant Feedback adjusts memory for subsequent executions, eliminating spills.

```sql
-- Check if database compatibility level enables IQP
ALTER DATABASE YourDB SET COMPATIBILITY_LEVEL = 150; -- SQL 2019

-- Batch Mode Memory Grant Feedback in action
-- First execution: optimizer estimates, may spill to tempdb
-- Subsequent executions: feedback adjusts the grant

-- Check for memory grant spills (evidence of MG feedback needed)
SELECT
    qs.execution_count,
    qs.total_spills,
    qs.avg_spills,   -- > 0 means spilling to tempdb
    SUBSTRING(qt.text, 1, 200) AS query_text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
WHERE qs.total_spills > 0
ORDER BY qs.total_spills DESC;

-- Table Variable Deferred Compilation (SQL 2019)
-- Old behavior: table variables always estimated as 1 row
-- New behavior: first-use compilation with actual count
DECLARE @Orders TABLE (OrderID INT, Total DECIMAL(10,2));
INSERT INTO @Orders SELECT OrderID, TotalAmount FROM Orders;

-- This join now uses actual row count (not 1) for plan
SELECT o.OrderID, c.CustomerName
FROM @Orders o
JOIN Customers c ON c.CustomerID = o.CustomerID;
```

SECTION 02

Advanced Indexing Strategies

02

CSD624 · Advanced Database Management in MS SQL

Dr A.J. Fofanah

Index Internals: B-Tree Architecture

Root → Intermediate → Leaf

B-Tree pages: Root holds high-level keys; leaf pages hold actual data (clustered) or row locators (non-clustered).

Page Size & Fill Factor

Each page = 8KB. Fill factor controls free space on leaf pages at rebuild time to reduce page splits.

Clustered Index

Defines physical row order on disk. Only one per table. Heap tables (no CI) have no physical order.

Non-Clustered Index

Separate B-tree with pointers back to the base table (RID for heaps, clustering key for CI tables).

Creating & Analyzing Clustered Indexes

Tips

The clustered index key is included in every non-clustered index as a row locator. Choosing a narrow, unique, ever-increasing key (e.g., IDENTITY INT) minimises page splits and keeps non-clustered index entries small. Wide or random clustered keys (GUID) cause fragmentation.

```sql
-- Create a well-designed clustered index
CREATE TABLE Sales.Orders (
    OrderID     INT IDENTITY(1,1) NOT NULL,
    CustomerID  INT NOT NULL,
    OrderDate   DATE NOT NULL,
    TotalAmount DECIMAL(12,2),
    CONSTRAINT PK_Orders PRIMARY KEY CLUSTERED (OrderID)
);

-- Inspect the index structure
SELECT
    i.name AS index_name,
    i.type_desc,
    i.is_unique,
    ic.key_ordinal,
    c.name AS column_name,
    ic.is_included_column
FROM sys.indexes i
JOIN sys.index_columns ic ON ic.object_id = i.object_id AND ic.index_id = i.index_id
JOIN sys.columns c        ON c.object_id  = ic.object_id AND c.column_id = ic.column_id
WHERE OBJECT_NAME(i.object_id) = 'Orders'
ORDER BY i.index_id, ic.key_ordinal;

-- Check fragmentation
SELECT index_id, avg_fragmentation_in_percent, page_count
FROM sys.dm_db_index_physical_stats(DB_ID(), OBJECT_ID('Sales.Orders'), NULL, NULL, 'DETAILED')
ORDER BY avg_fragmentation_in_percent DESC;
```

Covering Indexes & Included Columns

Tips

A covering index satisfies a query entirely from the index pages, eliminating a Key Lookup back to the base table. Include non-key columns using the INCLUDE clause — they live only in leaf pages and don't affect index depth. This is the single most impactful indexing technique for read-heavy workloads.

```
-- ❌ Without covering index: Key Lookup penalty
SELECT CustomerID, OrderDate, TotalAmount
FROM Sales.Orders
WHERE CustomerID = 42;
-- Plan shows: Index Seek on IX_CustomerID → Key Lookup (expensive!)

-- ✅ Covering index: eliminates Key Lookup
CREATE NONCLUSTERED INDEX IX_Orders_CustomerID_Covering
ON Sales.Orders (CustomerID)
INCLUDE (OrderDate, TotalAmount);
-- Plan now shows: Index Seek only — no Key Lookup!

-- Covering index for a specific report query
CREATE NONCLUSTERED INDEX IX_Orders_Date_Cust_Covering
ON Sales.Orders (OrderDate DESC, CustomerID)
INCLUDE (TotalAmount, Status, ShipCity)
WHERE Status IN ('Shipped', 'Delivered');  -- filtered too!

-- Validate via execution plan — look for "Index Seek" with no lookup
-- Also check sys.dm_db_index_usage_stats to confirm seeks vs scans
SELECT user_seeks, user_scans, user_lookups, last_user_seek
FROM sys.dm_db_index_usage_stats
WHERE object_id = OBJECT_ID('Sales.Orders') AND database_id = DB_ID();
```

Columnstore Indexes for Analytics

Tips

Columnstore indexes store data column-by-column rather than row-by-row, enabling extreme compression (5-10x) and batch-mode execution for aggregation queries. Ideal for data warehouse fact tables and reporting. SQL Server 2019 supports both clustered columnstore (full table) and non-clustered (updatable overlay).

```
-- Clustered Columnstore Index for a fact table
CREATE TABLE DW.FactSales (
    SaleDateKey  INT,
    CustomerKey  INT,
    ProductKey   INT,
    Quantity     INT,
    SalesAmount  DECIMAL(12,2),
    DiscountAmt  DECIMAL(12,2)
);

CREATE CLUSTERED COLUMNSTORE INDEX CCI_FactSales
ON DW.FactSales
WITH (MAXDOP = 4);

-- Non-clustered columnstore on an OLTP table (read-optimised overlay)
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Orders_Analytics
ON Sales.Orders (OrderDate, CustomerID, TotalAmount, Status);

-- Verify columnstore segment health
SELECT
    OBJECT_NAME(i.object_id) AS table_name,
    i.name AS index_name,
    css.row_count,
    css.size_in_bytes / 1024 AS size_kb,
    css.state_description,  -- OPEN, CLOSED, COMPRESSED
    css.trim_reason_desc
FROM sys.column_store_segments css
JOIN sys.partitions p ON p.partition_id = css.partition_id
JOIN sys.indexes i    ON i.object_id = p.object_id AND i.index_id = p.index_id
ORDER BY css.row_count DESC;
```

Filtered & Composite Indexes

Tips

Filtered indexes index only a subset of rows using a WHERE clause — smaller, faster, and require less maintenance. Ideal for sparse columns, status flags, or soft-deleted rows. Composite indexes follow the Equality-Equality-Range rule: put equality predicates first, then range predicates last.

```sql
-- Filtered index: only active, unshipped orders
CREATE NONCLUSTERED INDEX IX_Orders_Active
ON Sales.Orders (CustomerID, OrderDate)
INCLUDE (TotalAmount)
WHERE Status = 'Pending' AND IsDeleted = 0;
-- Index contains only ~5% of rows but answers 80% of queries!

-- Composite index: equality columns first, range last
-- Query: WHERE Region = 'EMEA' AND SalesRep = 101 AND SaleDate >= '2024-01-01'
CREATE NONCLUSTERED INDEX IX_Sales_Region_Rep_Date
ON Sales.SalesData (Region, SalesRepID, SaleDate)  -- equality, equality, range
INCLUDE (Revenue, Units);

-- Wrong order (range first — inhibits seek on later columns)
-- CREATE INDEX IX_Bad ON Sales.SalesData (SaleDate, Region, SalesRepID); ✗

-- Unique filtered index to enforce partial uniqueness
CREATE UNIQUE NONCLUSTERED INDEX UIX_Employee_ActiveEmail
ON HR.Employees (Email)
WHERE IsActive = 1;
-- Allows duplicate emails for terminated employees, but enforces
-- uniqueness among active ones
```

Index Fragmentation & Maintenance

Tips

Fragmentation occurs when index pages become out-of-order (external fragmentation) or sparsely filled (internal fragmentation). Rule of thumb: REORGANIZE for 10-30% fragmentation, REBUILD for >30%. REBUILD is an offline operation by default; use ONLINE = ON to allow concurrent access.

```sql
-- Measure fragmentation across all indexes
SELECT
    OBJECT_NAME(ips.object_id)         AS table_name,
    i.name                             AS index_name,
    ips.avg_fragmentation_in_percent,
    ips.page_count,
    ips.avg_page_space_used_in_percent
FROM sys.dm_db_index_physical_stats(
    DB_ID(), NULL, NULL, NULL, 'SAMPLED') ips
JOIN sys.indexes i
    ON i.object_id = ips.object_id AND i.index_id = ips.index_id
WHERE ips.page_count > 100  -- only meaningful-sized indexes
ORDER BY ips.avg_fragmentation_in_percent DESC;

-- Reorganize (online, minimal locking)
ALTER INDEX IX_Orders_CustomerID_Covering
ON Sales.Orders REORGANIZE;

-- Rebuild (offline by default; ONLINE option for Enterprise)
ALTER INDEX IX_Orders_CustomerID_Covering
ON Sales.Orders REBUILD
WITH (FILLFACTOR = 80, ONLINE = ON, MAXDOP = 2);

-- Rebuild ALL indexes on a table
ALTER INDEX ALL ON Sales.Orders
REBUILD WITH (FILLFACTOR = 85, ONLINE = ON);

-- Update statistics after rebuild
UPDATE STATISTICS Sales.Orders WITH FULLSCAN;
```

Missing Index DMVs

Tips

SQL Server tracks index opportunities encountered during query execution in the sys.dm_db_missing_index DMVs. These are starting points, not automatic recommendations. Always validate suggestions against actual workload patterns, existing indexes, and the equality/inequality column ordering rule.

```sql
-- Top missing indexes by potential impact
SELECT TOP 20
    mig.index_group_handle,
    mid.object_id,
    OBJECT_NAME(mid.object_id)              AS table_name,
    mid.equality_columns,
    mid.inequality_columns,
    mid.included_columns,
    migs.unique_compiles,
    migs.user_seeks,
    migs.avg_total_user_cost * migs.user_seeks AS impact_score
FROM sys.dm_db_missing_index_groups     mig
JOIN sys.dm_db_missing_index_group_stats migs
    ON migs.group_handle = mig.index_group_handle
JOIN sys.dm_db_missing_index_details    mid
    ON mid.index_handle  = mig.index_handle
WHERE mid.database_id = DB_ID()
ORDER BY impact_score DESC;

-- Generate CREATE INDEX scripts from suggestions
SELECT
    'CREATE NONCLUSTERED INDEX IX_' +
    OBJECT_NAME(mid.object_id) + '_' +
    REPLACE(ISNULL(mid.equality_columns,''), ', ','_') +
    ' ON ' + mid.statement +
    ' (' + ISNULL(mid.equality_columns,'') +
    CASE WHEN mid.inequality_columns IS NOT NULL
         THEN CASE WHEN mid.equality_columns IS NOT NULL
                   THEN ', ' ELSE '' END + mid.inequality_columns
         ELSE '' END + ')' +
    ISNULL(' INCLUDE (' + mid.included_columns + ')','') + ';'
FROM sys.dm_db_missing_index_details mid
JOIN sys.dm_db_missing_index_groups  mig ON mig.index_handle = mid.index_handle
WHERE mid.database_id = DB_ID();
```

Index Usage Statistics

Tips

sys.dm_db_index_usage_stats tracks seeks, scans, lookups, and updates since the last SQL Server restart. Unused indexes still consume write overhead, every INSERT, UPDATE, DELETE must maintain them. Regularly audit and drop confirmed-unused indexes to improve write throughput.

```sql
-- Find unused indexes (candidates for removal)
SELECT
    OBJECT_NAME(i.object_id)   AS table_name,
    i.name                     AS index_name,
    i.type_desc,
    ISNULL(ius.user_seeks, 0)  AS seeks,
    ISNULL(ius.user_scans, 0)  AS scans,
    ISNULL(ius.user_lookups,0) AS lookups,
    ISNULL(ius.user_updates,0) AS updates,   -- write cost
    ius.last_user_seek,
    ius.last_user_scan
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats ius
    ON ius.object_id = i.object_id
    AND ius.index_id = i.index_id
    AND ius.database_id = DB_ID()
WHERE i.type_desc = 'NONCLUSTERED'
    AND ISNULL(ius.user_seeks, 0) = 0
    AND ISNULL(ius.user_scans, 0) = 0
    AND ISNULL(ius.user_lookups, 0) = 0
    AND ISNULL(ius.user_updates, 0) > 100  -- being maintained!
ORDER BY ISNULL(ius.user_updates, 0) DESC;

-- Disable before dropping (safer — reversible)
ALTER INDEX IX_Orders_OldColumn ON Sales.Orders DISABLE;

-- Drop after confirming no impact
DROP INDEX IX_Orders_OldColumn ON Sales.Orders;
```

Partitioned Indexes & Partition Elimination

Tips

Table partitioning splits large tables into smaller, manageable segments by a partition function and scheme. Partition elimination means the optimizer only scans relevant partitions, dramatically reducing I/O. Aligned indexes (same partition scheme as the table) are required for partition switching.

```sql
-- Step 1: Partition function (split by year)
CREATE PARTITION FUNCTION PF_OrderDate (DATE)
AS RANGE RIGHT FOR VALUES
('2021-01-01','2022-01-01','2023-01-01','2024-01-01');

-- Step 2: Partition scheme (map to filegroups)
CREATE PARTITION SCHEME PS_OrderDate
AS PARTITION PF_OrderDate
ALL TO ([PRIMARY]);  -- or specify separate FGs per partition

-- Step 3: Create partitioned table
CREATE TABLE Sales.OrdersPartitioned (
    OrderID    INT IDENTITY,
    OrderDate  DATE NOT NULL,
    CustomerID INT,
    Total      DECIMAL(12,2),
    CONSTRAINT PK_OrdersP PRIMARY KEY CLUSTERED (OrderID, OrderDate)
) ON PS_OrderDate (OrderDate);

-- Step 4: Verify partition elimination in execution plan
SELECT COUNT(*), SUM(Total)
FROM Sales.OrdersPartitioned
WHERE OrderDate >= '2023-01-01' AND OrderDate < '2024-01-01';
-- Plan shows: Partition 4 only — others eliminated!

-- Check row distribution per partition
SELECT partition_number, row_count
FROM sys.dm_db_partition_stats
WHERE object_id = OBJECT_ID('Sales.OrdersPartitioned');
```

SECTION 03

Security, Encryption & Compliance

03

CSD624 · Advanced Database Management in MS SQL

Dr A.J. Fofanah

SQL Server Security Architecture

Authentication

Windows (Kerberos/NTLM) or SQL Server authentication. Contained databases allow DB-level users.

Authorisation

GRANT / DENY / REVOKE on securables. Principal hierarchy: Logins → Users → Roles → Permissions.

Encryption Layers

TDE (data at rest), Always Encrypted (column-level), TLS/SSL (data in transit), Backup Encryption.

Auditing & Compliance

SQL Server Audit, Extended Events, and Change Data Capture for GDPR/HIPAA evidence.

Transparent Data Encryption (TDE)

Tips

TDE encrypts the entire database at the page level using a Database Encryption Key (DEK) protected by a server certificate. Data files (.mdf/.ldf) and backups are encrypted transparently, no application changes needed. You must backup the certificate immediately after creation or risk permanent data loss.

```sql
-- Step 1: Create master key in master database
USE master;
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'Str0ng$ecretKey!2024';

-- Step 2: Create server certificate
CREATE CERTIFICATE TDE_Certificate
WITH SUBJECT = 'TDE for SalesDB',
EXPIRY_DATE = '2027-01-01';

-- Step 3: IMMEDIATELY backup certificate + private key
BACKUP CERTIFICATE TDE_Certificate
TO FILE = 'C:BackupTDE_Certificate.cer'
WITH PRIVATE KEY (
    FILE = 'C:BackupTDE_PrivateKey.pvk',
    ENCRYPTION BY PASSWORD = 'BackupKey!2024'
);

-- Step 4: Create DEK in target database
USE SalesDB;
CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_256
ENCRYPTION BY SERVER CERTIFICATE TDE_Certificate;

-- Step 5: Enable TDE
ALTER DATABASE SalesDB SET ENCRYPTION ON;

-- Monitor encryption progress
SELECT name, encryption_state,
    encryption_state_desc,
    percent_complete
FROM sys.dm_database_encryption_keys
JOIN sys.databases ON databases.database_id = dm_database_encryption_keys.database_id;
```

Always Encrypted — Column-Level

Explanation

Always Encrypted protects sensitive columns so that the SQL Server engine never sees the plaintext — encryption/decryption happens in the client driver. Two key types: Column Master Key (CMK, in Windows Certificate Store or Azure Key Vault) and Column Encryption Key (CEK, stored encrypted in SQL Server).

```
-- Step 1: Create Column Master Key metadata
CREATE COLUMN MASTER KEY CMK_AzureKeyVault
WITH (
    KEY_STORE_PROVIDER_NAME = 'AZURE_KEY_VAULT',
    KEY_PATH = 'https://myvault.vault.azure.net/keys/MyCMK/abc123'
);

-- Step 2: Create Column Encryption Key (CEK)
CREATE COLUMN ENCRYPTION KEY CEK_SensitiveData
WITH VALUES (
    COLUMN_MASTER_KEY = CMK_AzureKeyVault,
    ALGORITHM = 'RSA_OAEP',
    ENCRYPTED_VALUE = 0x01700000... -- generated by SSMS wizard
);

-- Step 3: Define encrypted columns
CREATE TABLE HR.Employees (
    EmployeeID  INT IDENTITY PRIMARY KEY,
    FullName    NVARCHAR(100),  -- not encrypted
    SSN         CHAR(11) ENCRYPTED WITH (
                    ENCRYPTION_TYPE = DETERMINISTIC,  -- allows equality search
                    ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256',
                    COLUMN_ENCRYPTION_KEY = CEK_SensitiveData),
    Salary      DECIMAL(10,2) ENCRYPTED WITH (
                    ENCRYPTION_TYPE = RANDOMIZED,     -- higher security, no search
                    ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256',
                    COLUMN_ENCRYPTION_KEY = CEK_SensitiveData)
);

-- Application must use Column Encryption Setting=Enabled in connection string
-- Server only ever stores/returns ciphertext for encrypted columns
```

Row-Level Security (RLS)

Tip

RLS enforces row-level access control using inline table-valued functions (predicates) as security policies. Filter predicates silently hide rows from SELECT. Block predicates prevent writes. The user never knows filtered rows exist, ideal for multi-tenant SaaS applications.

```
-- Step 1: Create the security predicate function
CREATE FUNCTION Security.fn_RegionAccess (@Region NVARCHAR(50))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN (
    SELECT 1 AS AccessGranted
    WHERE
        -- Admins see everything
        IS_MEMBER('db_admin') = 1
        OR
        -- Sales users see only their assigned region
        EXISTS (
            SELECT 1 FROM dbo.UserRegionMapping
            WHERE LoginName = USER_NAME()
              AND Region = @Region
        )
);
GO

-- Step 2: Create the security policy
CREATE SECURITY POLICY Sales.RegionSecurityPolicy
ADD FILTER PREDICATE Security.fn_RegionAccess(Region)
    ON Sales.Orders,
ADD BLOCK PREDICATE Security.fn_RegionAccess(Region)
    ON Sales.Orders AFTER INSERT
WITH (STATE = ON);

-- Test as a regional user
EXECUTE AS USER = 'emea_user';
SELECT * FROM Sales.Orders;  -- sees only EMEA rows
REVERT;

-- Disable for maintenance
ALTER SECURITY POLICY Sales.RegionSecurityPolicy WITH (STATE = OFF);
```

Dynamic Data Masking (DDM)

Tips

DDM masks sensitive data at query time for unauthorised users without modifying stored data. Four mask types: default (full mask), partial (show first/last N chars), email (a***@d***.com), and random (random number range). Privileged users with UNMASK permission see the real data.

```
-- Add masking to existing columns
ALTER TABLE HR.Employees
ALTER COLUMN SSN CHAR(11) MASKED WITH (FUNCTION = 'partial(0,"XXX-XX-",4)');
-- Shows: XXX-XX-6789

ALTER TABLE HR.Employees
ALTER COLUMN Email NVARCHAR(255) MASKED WITH (FUNCTION = 'email()');
-- Shows: aXXX@XXXX.com

ALTER TABLE HR.Employees
ALTER COLUMN Salary DECIMAL(10,2) MASKED WITH (FUNCTION = 'default()');
-- Shows: 0.00

ALTER TABLE HR.Employees
ALTER COLUMN PhoneExt INT MASKED WITH (FUNCTION = 'random(1000, 9999)');
-- Shows: random 4-digit number

-- Create restricted user (sees masked data)
CREATE USER MaskTestUser WITHOUT LOGIN;
GRANT SELECT ON HR.Employees TO MaskTestUser;

EXECUTE AS USER = 'MaskTestUser';
SELECT SSN, Email, Salary FROM HR.Employees;
-- SSN: XXX-XX-6789  Email: aXXX@XXXX.com  Salary: 0.00
REVERT;

-- Grant UNMASK to privileged users
GRANT UNMASK TO HRManager;
```

SQL Server Audit & Compliance

Tips

SQL Server Audit (Enterprise) records server and database-level events to a file, Windows Event Log, or Application Log. Required for SOX, PCI-DSS, HIPAA, and GDPR evidence. Server Audit Specifications capture login/logout events; Database Audit Specifications track DML and schema changes.

```
-- Step 1: Create the audit (writes to file)
CREATE SERVER AUDIT SalesDB_Audit
TO FILE (
    FILEPATH = 'C:SQLAudit',
    MAXSIZE   = 100 MB,
    MAX_FILES = 10,
    RESERVE_DISK_SPACE = OFF
)
WITH (QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE);

ALTER SERVER AUDIT SalesDB_Audit WITH (STATE = ON);

-- Step 2: Server Audit Specification (login events)
CREATE SERVER AUDIT SPECIFICATION AuditSpec_Logins
FOR SERVER AUDIT SalesDB_Audit
ADD (FAILED_LOGIN_GROUP),
ADD (SUCCESSFUL_LOGIN_GROUP),
ADD (LOGOUT_GROUP)
WITH (STATE = ON);

-- Step 3: Database Audit Specification (DML on sensitive tables)
USE SalesDB;
CREATE DATABASE AUDIT SPECIFICATION AuditSpec_SensitiveTables
FOR SERVER AUDIT SalesDB_Audit
ADD (SELECT, INSERT, UPDATE, DELETE ON HR.Employees BY PUBLIC),
ADD (SELECT ON Finance.SalaryGrades BY PUBLIC)
WITH (STATE = ON);

-- Query audit log
SELECT event_time, action_id, succeeded,
       server_principal_name, database_name, object_name, statement
FROM sys.fn_get_audit_file('C:SQLAudit*.sqlaudit', DEFAULT, DEFAULT)
ORDER BY event_time DESC;
```

Permissions Management & Least Privilege

Tips

The principle of least privilege grants only the minimum permissions required. Use database roles and schema-level permissions rather than per-table grants. EXECUTE AS allows stored procedures to run with elevated context without exposing underlying table permissions to the caller.

```
-- Create application role with minimal permissions
CREATE ROLE AppRole_ReadOrders;
GRANT SELECT ON SCHEMA::Sales TO AppRole_ReadOrders;
GRANT EXECUTE ON SCHEMA::Sales TO AppRole_ReadOrders;

-- Grant insert/update only on specific tables
CREATE ROLE AppRole_OrderEntry;
GRANT INSERT, UPDATE ON Sales.Orders TO AppRole_OrderEntry;
GRANT INSERT ON Sales.OrderDetails TO AppRole_OrderEntry;
DENY DELETE ON Sales.Orders TO AppRole_OrderEntry;

-- Assign roles to users
ALTER ROLE AppRole_ReadOrders ADD MEMBER WebAppUser;
ALTER ROLE AppRole_OrderEntry ADD MEMBER OrderClerkUser;

-- EXECUTE AS for privilege escalation in stored procs
CREATE PROCEDURE Sales.usp_GetSensitiveOrderData
WITH EXECUTE AS 'ProcOwner'  -- runs as ProcOwner, not caller
AS
BEGIN
    SELECT o.OrderID, c.SSN_Last4
    FROM Sales.Orders o
    JOIN HR.Customers c ON c.CustomerID = o.CustomerID;
END;
-- Caller needs EXECUTE on the proc only, not on HR.Customers!

-- Audit current permissions
SELECT
    pr.name AS principal_name,
    pr.type_desc,
    pe.state_desc,
    pe.permission_name,
    ISNULL(OBJECT_NAME(pe.major_id), 'DB-Level') AS object_name
FROM sys.database_permissions pe
JOIN sys.database_principals pr ON pr.principal_id = pe.grantee_principal_id
ORDER BY pr.name, pe.permission_name;
```

Change Data Capture (CDC)

Tips

CDC captures row-level INSERT, UPDATE, DELETE changes to tracked tables and stores them in change tables. Used for GDPR right-to-erasure audit trails, ETL pipeline delta loads, and compliance reporting. Changes include before/after images and an operation column (1=delete, 2=insert, 3=update before, 4=update after).

```
-- Enable CDC on the database
EXEC sys.sp_cdc_enable_db;

-- Enable CDC on a specific table
EXEC sys.sp_cdc_enable_table
    @source_schema    = N'Sales',
    @source_name      = N'Orders',
    @role_name        = N'cdc_reader',  -- role with read access
    @captured_column_list = N'OrderID,CustomerID,TotalAmount,Status',
    @supports_net_changes = 1;

-- Query changes in a time range
DECLARE @from_lsn BINARY(10) = sys.fn_cdc_map_time_to_lsn('smallest greater than or equal', '2024-01-01');
DECLARE @to_lsn   BINARY(10) = sys.fn_cdc_get_max_lsn();

SELECT
    __$operation,       -- 1=Del 2=Ins 3=Before 4=After
    __$update_mask,
    OrderID, CustomerID, TotalAmount, Status
FROM cdc.fn_cdc_get_all_changes_Sales_Orders
    (@from_lsn, @to_lsn, 'all with mask')
ORDER BY __$start_lsn;

-- Net changes only (final state per key)
SELECT __$operation, OrderID, TotalAmount, Status
FROM cdc.fn_cdc_get_net_changes_Sales_Orders
    (@from_lsn, @to_lsn, 'all with mask');
```

Backup Encryption & Data-at-Rest

Tips

Unencrypted backups are a major compliance risk — anyone with the .bak file can restore it. SQL Server native backup encryption uses AES 128/256. Combined with TDE, this ensures both the live database and all backups are protected. Always store encryption certificates in a separate, secure location.

```
-- Encrypted backup using existing TDE certificate
BACKUP DATABASE SalesDB
TO DISK = 'D:BackupsSalesDB_encrypted.bak'
WITH
    ENCRYPTION (
        ALGORITHM = AES_256,
        SERVER CERTIFICATE = TDE_Certificate
    ),
    COMPRESSION,
    STATS = 10;

-- Encrypted differential backup
BACKUP DATABASE SalesDB
TO DISK = 'D:BackupsSalesDB_diff.bak'
WITH DIFFERENTIAL,
    ENCRYPTION (ALGORITHM = AES_256, SERVER CERTIFICATE = TDE_Certificate),
    COMPRESSION;

-- Encrypted transaction log backup
BACKUP LOG SalesDB
TO DISK = 'D:BackupsSalesDB_log.bak'
WITH ENCRYPTION (ALGORITHM = AES_256, SERVER CERTIFICATE = TDE_Certificate);

-- Verify backup integrity
RESTORE VERIFYONLY
FROM DISK = 'D:BackupsSalesDB_encrypted.bak'
WITH LOADHISTORY;

-- Check backup encryption metadata
SELECT backup_start_date, type, key_algorithm, encryptor_type, encryptor_thumbprint
FROM msdb.dbo.backupset
WHERE database_name = 'SalesDB'
ORDER BY backup_start_date DESC;
```

SECTION 04

High Availability & Cloud Integration

04

CSD624 · Advanced Database Management in MS SQL

Dr A.J. Fofanah

HA/DR Concepts & RTO/RPO

RTO: Recovery Time Objective

Maximum acceptable downtime. Always On AG automatic failover: seconds. Log Shipping: minutes to hours.

RPO: Recovery Point Objective

Maximum acceptable data loss. Synchronous AG replicas: zero data loss. Asynchronous: seconds to minutes.

Always On AG vs FCI

AG: database-level HA, readable secondaries, no shared storage. FCI: instance-level HA, shared SAN, no secondary reads.

Azure HA Options

Azure SQL MI Business Critical: built-in AG. Geo-replication: async cross-region. Azure SQL Hyperscale: rapid scale-out.

Always On Availability Groups — Setup

Tips

Always On AGs provide database-level redundancy without shared storage. Primary replica handles read/write; secondary replicas can serve read-only queries, offloading reporting workloads. WSFC (Windows Server Failover Cluster) underpins the health detection and automatic failover mechanism.

```
-- Step 1: Enable Always On on the SQL instance (via PowerShell)
-- Enable-SqlAlwaysOn -ServerInstance "SQL01" -Force

-- Step 2: Create the Availability Group
CREATE AVAILABILITY GROUP AG_SalesDB
WITH (
    AUTOMATED_BACKUP_PREFERENCE = SECONDARY,  -- backups on secondary
    FAILURE_CONDITION_LEVEL = 3,
    HEALTH_CHECK_TIMEOUT = 30000
)
FOR DATABASE SalesDB
REPLICA ON
    'SQL01' WITH (
        ENDPOINT_URL = 'TCP://SQL01.domain.com:5022',
        AVAILABILITY_MODE = SYNCHRONOUS_COMMIT,  -- zero data loss
        FAILOVER_MODE = AUTOMATIC,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY)
    ),
    'SQL02' WITH (
        ENDPOINT_URL = 'TCP://SQL02.domain.com:5022',
        AVAILABILITY_MODE = SYNCHRONOUS_COMMIT,
        FAILOVER_MODE = AUTOMATIC,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY)
    ),
    'SQL03' WITH (
        ENDPOINT_URL = 'TCP://SQL03.domain.com:5022',
        AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT,  -- DR replica
        FAILOVER_MODE = MANUAL,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY)
    );

-- Create listener (virtual network name for client redirection)
ALTER AVAILABILITY GROUP AG_SalesDB
ADD LISTENER 'AG Listener' (WITH IP (('10.0.0.50','255.255.255.0')), PORT = 1433);
```

Monitoring Always On AGs

Tips

DMVs provide real-time health data for AG replicas and databases. Monitor synchronisation lag (redo_queue_size, log_send_queue_size) to understand your actual RPO. The AG dashboard in SSMS shows a quick visual overview, but DMV queries are essential for alerting and automation.

```sql
-- AG replica health and synchronisation state
SELECT
    ag.name             AS ag_name,
    ar.replica_server_name,
    ar.availability_mode_desc,
    ar.failover_mode_desc,
    ars.role_desc,
    ars.synchronization_health_desc,
    ars.connected_state_desc,
    ars.operational_state_desc
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar ON ar.group_id = ag.group_id
JOIN sys.dm_hadr_availability_replica_states ars
    ON ars.replica_id = ar.replica_id;

-- Database-level synchronisation lag
SELECT
    ag.name               AS ag_name,
    adb.database_name,
    drs.synchronization_state_desc,
    drs.synchronization_health_desc,
    drs.log_send_queue_size  / 1024 AS send_queue_mb,  -- RPO indicator
    drs.redo_queue_size      / 1024 AS redo_queue_mb,
    drs.log_send_rate        / 1024 AS send_rate_mb_s,
    drs.redo_rate            / 1024 AS redo_rate_mb_s,
    drs.last_hardened_time,
    drs.last_redone_time
FROM sys.availability_groups ag
JOIN sys.availability_databases_cluster adb ON adb.group_id = ag.group_id
JOIN sys.dm_hadr_database_replica_states  drs
    ON drs.group_database_id = adb.group_database_id
WHERE drs.is_local = 0;  -- secondary replicas only
```

Read-Only Routing for Secondaries

Tips

Read-only routing directs read-intent connections to a secondary replica automatically, transparently offloading reports and analytics. The application uses ApplicationIntent=ReadOnly in the connection string pointing to the AG listener. SQL Server routes it to a designated read-only replica.

```
-- Configure read-only routing URLs on each replica
ALTER AVAILABILITY GROUP AG_SalesDB
MODIFY REPLICA ON 'SQL01' WITH (
    PRIMARY_ROLE (READ_ONLY_ROUTING_LIST = ('SQL02','SQL03'))
);
ALTER AVAILABILITY GROUP AG_SalesDB
MODIFY REPLICA ON 'SQL02' WITH (
    SECONDARY_ROLE (READ_ONLY_ROUTING_URL = 'TCP://SQL02.domain.com:1433')
);
ALTER AVAILABILITY GROUP AG_SalesDB
MODIFY REPLICA ON 'SQL03' WITH (
    SECONDARY_ROLE (READ_ONLY_ROUTING_URL = 'TCP://SQL03.domain.com:1433')
);

-- Application connection string (routed to secondary automatically):
-- Server=AG_Listener,1433;Database=SalesDB;
-- Integrated Security=True;ApplicationIntent=ReadOnly;

-- Verify routing is working
SELECT
    @@SERVERNAME            AS connected_server,
    DATABASEPROPERTYEX(DB_NAME(),'Updateability') AS updateability;
-- Returns: connected_server=SQL02, updateability=READ_ONLY

-- Monitor secondary query load
SELECT session_id, login_name, host_name, program_name, reads, cpu_time
FROM sys.dm_exec_sessions
WHERE is_user_process = 1
ORDER BY reads DESC;
```

Log Shipping Configuration

Tips

Log shipping is a simpler HA alternative — transaction log backups are copied and restored to a warm standby server on a scheduled basis. RPO equals the log backup frequency (typically 15-60 min). The monitor server tracks latency and raises alerts when thresholds are breached.

```
-- Primary server: enable log shipping for the database
EXEC master.dbo.sp_add_log_shipping_primary_database
    @database               = N'SalesDB',
    @backup_directory       = N'\\FileServer\LogShipBackup\',
    @backup_share           = N'\\FileServer\LogShipBackup\',
    @backup_job_name        = N'LSBackup_SalesDB',
    @backup_retention_period = 4320,    -- 3 days in minutes
    @backup_threshold       = 60,       -- alert if no backup > 60 min
    @threshold_alert_enabled = 1,
    @history_retention_period = 5760,
    @backup_job_id          = @LS_BackupJobId OUTPUT,
    @primary_id             = @LS_PrimaryId  OUTPUT;

-- Secondary server: configure restore job
EXEC master.dbo.sp_add_log_shipping_secondary_primary
    @primary_server  = N'SQL01',
    @primary_database = N'SalesDB',
    @backup_source_directory  = N'\\FileServer\LogShipBackup\',
    @backup_destination_directory = N'D:\LogShipRestore\',
    @copy_job_name    = N'LSCopy_SalesDB',
    @restore_job_name = N'LSRestore_SalesDB',
    @file_retention_period = 4320;

-- Monitor log shipping status
SELECT primary_server, primary_database, secondary_server,
    last_backup_file, last_backup_date,
    last_copied_file, last_copied_date,
    last_restored_file, last_restored_date,
    restore_threshold  -- minutes before alert
FROM msdb.dbo.log_shipping_monitor_secondary;
```

Azure SQL Managed Instance Migration

Tips

Azure SQL MI offers near-100% SQL Server compatibility in a fully managed PaaS environment. The Database Migration Service (DMS) performs online migration with minimal downtime using log shipping-style continuous log replay. Applications pointing to MI use the same T-SQL code with no rewrites.

```
-- Pre-migration: assess compatibility using DMA tool output
-- or query directly for unsupported features

-- Check for features not supported in Azure SQL MI
SELECT DISTINCT
    f.name AS unsupported_feature
FROM sys.dm_db_persisted_sku_features f
-- Common: cross-db queries, linked servers (limited), CLR strict security

-- Online migration: Database Migration Service (DMS) uses LRS
-- Enable Log Replay Service on MI (receives log backups from on-prem)

-- Step 1: Start Log Replay Service migration
-- az sql midb log-replay start
--   --resource-group RG-Production
--   --managed-instance MI-SalesDB
--   --name SalesDB
--   --auto-complete
--   --last-backup-name "SalesDB_20240101_FULL.bak"

-- On SQL Server: take tail-log backup to finalise cutover
BACKUP LOG SalesDB
TO DISK = 'D:MigrationSalesDB_taillog.bak'
WITH NORECOVERY;  -- puts DB in restoring state

-- Post-migration verification on MI
SELECT @@VERSION;   -- Confirms Azure SQL MI
SELECT name, state_desc, compatibility_level
FROM sys.databases WHERE name = 'SalesDB';

-- Connection string for applications (MI endpoint):
-- Server=mi-salesdb.xxxx.database.windows.net;
-- Database=SalesDB;Authentication=Active Directory Integrated;
```

Monitoring with DMVs & Query Store

Tips

Query Store (introduced SQL 2016) is the 'flight recorder' for query plans — it persists plan and runtime statistics across restarts and plan changes. Essential for identifying plan regressions after upgrades or statistics changes. DMV-based monitoring complements it for real-time waits analysis.

```
-- Enable Query Store
ALTER DATABASE SalesDB SET QUERY_STORE = ON;
ALTER DATABASE SalesDB SET QUERY_STORE (
    OPERATION_MODE = READ_WRITE,
    CLEANUP_POLICY = (STALE_QUERY_THRESHOLD_DAYS = 30),
    DATA_FLUSH_INTERVAL_SECONDS = 900,
    INTERVAL_LENGTH_MINUTES = 60,
    MAX_STORAGE_SIZE_MB = 1024,
    QUERY_CAPTURE_MODE = AUTO  -- captures significant queries only
);

-- Find top regressed queries (plan changed + performance degraded)
SELECT TOP 10
    qsq.query_id,
    qsqt.query_sql_text,
    qsp.plan_id,
    qsrs.avg_duration / 1000 AS avg_ms,
    qsrs.avg_logical_io_reads
FROM sys.query_store_query        qsq
JOIN sys.query_store_query_text   qsqt ON qsqt.query_text_id = qsq.query_text_id
JOIN sys.query_store_plan         qsp  ON qsp.query_id = qsq.query_id
JOIN sys.query_store_runtime_stats qsrs ON qsrs.plan_id = qsp.plan_id
ORDER BY qsrs.avg_duration DESC;

-- Force a known-good plan
EXEC sys.sp_query_store_force_plan @query_id = 42, @plan_id = 7;

-- Top wait statistics (real-time)
SELECT TOP 10 wait_type, waiting_tasks_count,
    wait_time_ms / 1000.0 AS wait_s,
    signal_wait_time_ms / 1000.0 AS signal_wait_s
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN (-- filter benign waits
    'SLEEP_TASK','BROKER_TO_FLUSH','DISPATCHER_QUEUE_SEMAPHORE')
ORDER BY wait_time_ms DESC;
```

Temporal Tables — Time-Travel Queries

Tips

System-versioned temporal tables automatically track the full history of row changes, storing previous versions in a history table. Query data as of any past point in time using FOR SYSTEM_TIME clauses. Ideal for audit trails, slowly-changing dimension management, and 'what-was-the-state-at?' queries.

```sql
-- Create a temporal table
CREATE TABLE Sales.ProductPrices (
    ProductID    INT NOT NULL PRIMARY KEY,
    ProductName NVARCHAR(200),
    UnitPrice    DECIMAL(10,2),
    SupplierID   INT,
    -- System-period columns (managed by SQL Server)
    ValidFrom    DATETIME2 GENERATED ALWAYS AS ROW START NOT NULL,
    ValidTo      DATETIME2 GENERATED ALWAYS AS ROW END   NOT NULL,
    PERIOD FOR SYSTEM_TIME (ValidFrom, ValidTo)
)
WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE = Sales.ProductPricesHistory));

-- Normal DML — history tracked automatically
UPDATE Sales.ProductPrices SET UnitPrice = 29.99 WHERE ProductID = 101;
UPDATE Sales.ProductPrices SET UnitPrice = 34.99 WHERE ProductID = 101;

-- Query current state
SELECT * FROM Sales.ProductPrices WHERE ProductID = 101;

-- Time-travel: what was the price on a specific date?
SELECT ProductID, ProductName, UnitPrice, ValidFrom, ValidTo
FROM Sales.ProductPrices
FOR SYSTEM_TIME AS OF '2024-06-15 12:00:00'
WHERE ProductID = 101;

-- All price changes for a product (full history)
SELECT ProductID, UnitPrice, ValidFrom, ValidTo
FROM Sales.ProductPrices
FOR SYSTEM_TIME ALL
WHERE ProductID = 101
ORDER BY ValidFrom;
```

In-Memory OLTP (Hekaton)

Tips

In-Memory OLTP stores tables entirely in RAM using a lock-free, latch-free, optimistic concurrency model. Eliminates lock/latch contention that caps throughput on hot tables. Natively compiled stored procedures compile T-SQL to machine code, achieving microsecond execution for simple OLTP operations.

```
-- Create memory-optimized filegroup (required)
ALTER DATABASE SalesDB ADD FILEGROUP FG_MemoryOptimized
CONTAINS MEMORY_OPTIMIZED_DATA;

ALTER DATABASE SalesDB ADD FILE (
    NAME = 'SalesDB_MemOpt',
    FILENAME = 'D:DataSalesDB_MemOpt'
) TO FILEGROUP FG_MemoryOptimized;

-- Create memory-optimized table (DURABILITY options)
CREATE TABLE Sales.HotOrders (
    OrderID    INT NOT NULL CONSTRAINT PK_HotOrders PRIMARY KEY
                   NONCLUSTERED HASH WITH (BUCKET_COUNT = 1000000),
    CustomerID INT NOT NULL,
    OrderDate  DATETIME2 NOT NULL,
    Total      DECIMAL(12,2),
    Status     NVARCHAR(20)
)
WITH (MEMORY_OPTIMIZED = ON,
      DURABILITY = SCHEMA_AND_DATA);  -- persisted to disk

-- Natively compiled stored procedure (compiles to DLL)
CREATE PROCEDURE Sales.usp_InsertHotOrder
    @OrderID INT, @CustomerID INT, @Total DECIMAL(12,2)
WITH NATIVE_COMPILATION, SCHEMABINDING
AS BEGIN ATOMIC WITH (
    TRANSACTION ISOLATION LEVEL = SNAPSHOT,
    LANGUAGE = N'English'
)
    INSERT INTO Sales.HotOrders (OrderID, CustomerID, OrderDate, Total, Status)
    VALUES (@OrderID, @CustomerID, SYSDATETIME(), @Total, N'Pending');
END;
```

SECTION 05

Advanced T-SQL Patterns

05

CSD624 · Advanced Database Management in MS SQL

Dr A.J. Fofanah

Window Functions — Advanced Patterns

Tips

Window functions compute aggregate/ranking values across a 'window' of rows related to the current row, without collapsing the result set like GROUP BY. The OVER() clause defines the partition and ordering. Essential for running totals, moving averages, gaps-and-islands, and rank-based filtering.

```sql
-- Running total and moving average
SELECT
    OrderDate,
    SalesAmount,
    SUM(SalesAmount) OVER (ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
    AVG(SalesAmount) OVER (ORDER BY OrderDate
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)          AS moving_avg_7day
FROM Sales.DailySales;

-- LEAD/LAG: compare to previous/next row
SELECT
    OrderDate,
    SalesAmount,
    LAG(SalesAmount, 1, 0) OVER (ORDER BY OrderDate)  AS prev_day,
    LEAD(SalesAmount, 1, 0) OVER (ORDER BY OrderDate) AS next_day,
    SalesAmount - LAG(SalesAmount,1,0) OVER (ORDER BY OrderDate) AS day_over_day
FROM Sales.DailySales;

-- NTILE: divide into quartiles for segmentation
SELECT CustomerID, TotalSpend,
    NTILE(4) OVER (ORDER BY TotalSpend DESC) AS spend_quartile
FROM Sales.CustomerSummary;

-- Rank vs Dense_Rank vs Row_Number
SELECT ProductID, CategoryID, Revenue,
    RANK()       OVER (PARTITION BY CategoryID ORDER BY Revenue DESC) AS rnk,
    DENSE_RANK() OVER (PARTITION BY CategoryID ORDER BY Revenue DESC) AS dense_rnk,
    ROW_NUMBER() OVER (PARTITION BY CategoryID ORDER BY Revenue DESC) AS row_num
FROM Sales.ProductRevenue;
```

CTEs, Recursive Queries & Hierarchies

Tips

Common Table Expressions (CTEs) improve readability and enable recursive queries. Recursive CTEs traverse hierarchical structures (org charts, bill of materials, category trees) by defining an anchor member and a recursive member that references the CTE itself. The MAXRECURSION option prevents infinite loops.

```
-- Non-recursive CTE for readability
WITH
MonthlyRevenue AS (
    SELECT
        YEAR(OrderDate) AS yr,
        MONTH(OrderDate) AS mth,
        SUM(TotalAmount) AS revenue
    FROM Sales.Orders
    GROUP BY YEAR(OrderDate), MONTH(OrderDate)
),
RevenueWithRank AS (
    SELECT *, RANK() OVER (PARTITION BY yr ORDER BY revenue DESC) AS mth_rank
    FROM MonthlyRevenue
)
SELECT * FROM RevenueWithRank WHERE mth_rank <= 3;

-- Recursive CTE: traverse employee hierarchy
WITH OrgChart AS (
    -- Anchor: top-level (CEO)
    SELECT EmployeeID, FullName, ManagerID, 0 AS Level,
           CAST(FullName AS NVARCHAR(500)) AS HierarchyPath
    FROM HR.Employees WHERE ManagerID IS NULL

    UNION ALL

    -- Recursive: employees reporting to someone in OrgChart
    SELECT e.EmployeeID, e.FullName, e.ManagerID, oc.Level + 1,
           CAST(oc.HierarchyPath + ' > ' + e.FullName AS NVARCHAR(500))
    FROM HR.Employees e
    JOIN OrgChart oc ON oc.EmployeeID = e.ManagerID
)
SELECT Level, HierarchyPath, FullName
FROM OrgChart
ORDER BY HierarchyPath
OPTION (MAXRECURSION 20);
```

Tips

Dynamic SQL builds query strings at runtime, necessary for dynamic column lists, pivot operations, and multi-tenant query routing. Always use sp_executesql with parameterised queries, never string concatenation of user input (SQL injection). The parameterised approach also allows plan reuse.

```
-- ❌ Vulnerable to SQL injection — NEVER do this
DECLARE @tbl NVARCHAR(100) = 'Orders'; -- could be "Orders; DROP TABLE..."
EXEC ('SELECT * FROM ' + @tbl);

-- ✅ Safe: parameterise the value (not object names)
DECLARE @CustomerID INT = 42;
DECLARE @sql NVARCHAR(500) =
    N'SELECT OrderID, TotalAmount FROM Sales.Orders WHERE CustomerID = @CID';

EXEC sp_executesql @sql,
    N'@CID INT',   -- parameter definition
    @CID = @CustomerID;

-- ✅ Dynamic PIVOT: build column list from data
DECLARE @cols NVARCHAR(MAX), @pivot_sql NVARCHAR(MAX);

SELECT @cols = STRING_AGG(
    QUOTENAME(YEAR(OrderDate)), ', ')
    WITHIN GROUP (ORDER BY YEAR(OrderDate))
FROM (SELECT DISTINCT OrderDate FROM Sales.Orders) d;

SET @pivot_sql = N'
SELECT CustomerID, ' + @cols + '
FROM (
    SELECT CustomerID, YEAR(OrderDate) AS yr, TotalAmount
    FROM Sales.Orders
) src
PIVOT (SUM(TotalAmount) FOR yr IN (' + @cols + N')) pvt
ORDER BY CustomerID;';

EXEC sp_executesql @pivot_sql;
```

Error Handling & Transactions

Tips

Robust stored procedures use TRY/CATCH with explicit transaction management. Always check XACT_STATE() before attempting a ROLLBACK — some errors leave the transaction in a doomed, un-committable state. Log errors to a dedicated error table for post-incident analysis.

```
-- Robust stored procedure with full error handling
CREATE PROCEDURE Sales.usp_TransferOrder
    @OrderID INT,
    @ToCustomerID INT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;  -- automatically rollback on error

    BEGIN TRANSACTION;
    BEGIN TRY

        -- Validate source order exists and is active
        IF NOT EXISTS (SELECT 1 FROM Sales.Orders
                       WHERE OrderID = @OrderID AND Status = 'Active')
            THROW 50001, 'Order not found or not transferable.', 1;

        UPDATE Sales.Orders
            SET CustomerID = @ToCustomerID, ModifiedDate = SYSDATETIME()
        WHERE OrderID = @OrderID;

        -- Log the transfer
        INSERT INTO Audit.OrderTransfers (OrderID, ToCustomerID, TransferredAt, TransferredBy)
        VALUES (@OrderID, @ToCustomerID, SYSDATETIME(), SYSTEM_USER);

        COMMIT TRANSACTION;

    END TRY
    BEGIN CATCH
        IF XACT_STATE() <> 0  -- transaction exists (may be doomed)
            ROLLBACK TRANSACTION;

        -- Log to error table
        INSERT INTO dbo.ErrorLog (ErrorNumber, ErrorMessage, ErrorLine, Procedure, LogTime)
        VALUES (ERROR_NUMBER(), ERROR_MESSAGE(), ERROR_LINE(), ERROR_PROCEDURE(), SYSDATETIME());

        THROW;  -- re-raise to caller
    END CATCH;
END;
```

LEARNING OUTCOMES

By the End of CSD624, You Will Be Able To:

1. Design and interpret SQL Server execution plans; tune queries using statistics, hints, and AQP
2. Implement advanced indexing strategies — covering, columnstore, filtered, and partitioned indexes
3. Secure databases with TDE, Always Encrypted, Row-Level Security, DDM, and SQL Server Audit
4. Configure Always On AGs, log shipping, and migrate workloads to Azure SQL Managed Instance
5. Write advanced T-SQL: window functions, recursive CTEs, dynamic SQL, and robust error handling

Assessment: Lab + Project + Exam

Weekly Lab Sessions Required

Thank You, Question?

Dr A.J. Fofanah

CSD624 — Advanced Database Management in MS SQL

Master of Science in Computer Science